

React DevTools

Введение в React DevTools

React DevTools — это расширение для браузера, которое позволяет инспектировать иерархию компонентов React, просматривать и изменять их props и state, а также анализировать производительность приложения. Это незаменимый инструмент для разработки и отладки React-приложений.

Установка React DevTools

React DevTools доступен как расширение для популярных браузеров:

- [Chrome](#)
- [Firefox](#)
- [Edge](#)

Для использования React DevTools с другими браузерами или в мобильных приложениях, можно установить отдельный пакет:

```
npm install --save-dev react-devtools  
  
# или с использованием yarn  
yarn add --dev react-devtools
```

После установки пакета, вы можете запустить DevTools как отдельное приложение:

```
npx react-devtools
```

Основные возможности React DevTools

Панель Components

Панель Components позволяет исследовать дерево компонентов React и просматривать их props, state и hooks.

Исследование дерева компонентов

В левой части панели отображается иерархия компонентов. Вы можете:

- Щелкнуть на компонент, чтобы просмотреть его детали
- Использовать стрелки для навигации по дереву
- Искать компоненты по имени с помощью поля поиска
- Фильтровать компоненты по типу (например, показывать только компоненты с хуками)

Просмотр props и state

В правой части панели отображаются props и state выбранного компонента. Вы можете:

- Просматривать значения props и state
- Редактировать значения props и state (для отладки)
- Копировать значения в буфер обмена
- Переключаться между различными форматами отображения (например, показывать или скрывать строковые кавычки)

Просмотр хуков

Для функциональных компонентов, которые используют хуки, React DevTools показывает информацию о каждом хуке:

- **useState**: текущее значение и функция для обновления
- **useReducer**: текущее состояние и функция dispatch
- **useContext**: текущее значение контекста
- **useRef**: текущее значение ref
- **useMemo** и **useCallback**: мемоизированные значения и функции
- **useEffect** и **useLayoutEffect**: зависимости эффекта

Панель Profiler

Панель Profiler позволяет анализировать производительность приложения, записывая и визуализируя рендеринг компонентов.

Запись профиля

Для записи профиля:

1. Нажмите кнопку запуска записи (круглая кнопка)
2. Выполните действия в приложении, которые вы хотите проанализировать
3. Нажмите кнопку остановки записи

Анализ профиля

После записи профиля, вы можете:

- Просматривать коммиты (обновления) в виде диаграммы
- Выбирать отдельные коммиты для детального анализа
- Видеть, какие компоненты были перерендерены и сколько времени это заняло
- Определять, какие компоненты вызывают наибольшие задержки

Режимы отображения

Profiler предлагает несколько режимов отображения:

- **Flame Chart:** показывает иерархию компонентов в виде пламенной диаграммы, где ширина блока пропорциональна времени рендеринга
- **Ranked Chart:** показывает компоненты, отсортированные по времени рендеринга
- **Component Chart:** показывает, сколько раз каждый компонент был перерендерен
- **Interactions:** показывает взаимодействия пользователя и связанные с ними обновления (если используется трассировка взаимодействий)

Продвинутые возможности

Компоненты-хосты и компоненты-волокна

React DevTools различает два типа компонентов:

- **Компоненты-хосты:** DOM-элементы, такие как `div`, `span` и т.д.
- **Компоненты-волокна:** пользовательские компоненты React

Вы можете фильтровать дерево компонентов, чтобы показывать только компоненты-волокна, что упрощает навигацию по крупным приложениям.

Инспектирование DOM-элементов

Вы можете щелкнуть правой кнопкой мыши на компонент в дереве и выбрать "Inspect DOM element", чтобы перейти к соответствующему DOM-элементу в панели Elements браузера.

Подсветка компонентов

Вы можете включить подсветку компонентов на странице, нажав на значок глаза в панели инструментов. Это позволяет видеть, какие DOM-элементы соответствуют выбранному компоненту.

Сохранение и загрузка профилей

Вы можете сохранять профили производительности в файл и загружать их позже для анализа или для сравнения с другими профилями.

Настройки

React DevTools предлагает различные настройки, которые можно изменить, нажав на значок шестеренки:

- **General:** общие настройки, такие как тема (светлая или темная)
- **Components:** настройки отображения компонентов, такие как сортировка и фильтрация
- **Profiler:** настройки профилировщика, такие как запись причин рендеринга

Отладка с помощью React DevTools

Поиск проблем с производительностью

1. Запустите Profiler и запишите взаимодействие, которое кажется медленным
2. Проанализируйте, какие компоненты перерендериваются и сколько времени это занимает
3. Определите компоненты, которые перерендериваются без необходимости

4. Оптимизируйте эти компоненты с помощью `React.memo`, `useMemo` или `useCallback`

Отладка проблем с рендерингом

1. Найдите компонент, который не отображается или отображается неправильно
2. Проверьте его props и state, чтобы убедиться, что они имеют ожидаемые значения
3. Если значения неправильные, проследите цепочку props до источника данных
4. Используйте подсветку компонентов, чтобы убедиться, что компонент рендерится в ожидаемом месте

Отладка проблем с хуками

1. Найдите компонент, который использует хуки
2. Проверьте значения хуков, чтобы убедиться, что они имеют ожидаемые значения
3. Для `useEffect` и `useLayoutEffect`, проверьте зависимости, чтобы убедиться, что эффект запускается в нужное время
4. Для `useMemo` и `useCallback`, проверьте зависимости, чтобы убедиться, что мемоизация работает правильно

Интеграция с другими инструментами

Redux DevTools

Если ваше приложение использует Redux, вы можете установить [Redux DevTools](#) для отладки состояния Redux. React DevTools и Redux DevTools хорошо работают вместе, позволяя отлаживать как компоненты, так и состояние приложения.

React Router DevTools

Если ваше приложение использует React Router, вы можете установить [React Router DevTools](#) для отладки маршрутизации.

Советы и рекомендации

Именованное компонентов

Используйте осмысленные имена для компонентов, чтобы упростить их поиск в React DevTools. Анонимные компоненты и компоненты высшего порядка могут быть трудно идентифицировать.

```
// Плохо
export default () => <div>Привет</div>;

// Хорошо
export default function Greeting() {
  return <div>Привет</div>;
}
```

Использование displayName

Для компонентов, созданных с помощью `React.forwardRef`, `React.memo` или других API, которые не сохраняют имя функции, используйте свойство `displayName` для улучшения отладки.

```
const MyComponent = React.forwardRef((props, ref) => {  
  return <div ref={ref}>{props.children}</div>;  
});  
  
MyComponent.displayName = 'MyComponent';
```

Трассировка взаимодействий

Вы можете использовать API трассировки взаимодействий для отслеживания, какие обновления вызваны конкретными взаимодействиями пользователя.

```
import { unstable_trace as trace } from 'scheduler/tracing';  
  
function handleClick() {  
  trace('button click', performance.now(), () => {  
    this.setState({ count: this.state.count + 1 });  
  });  
}
```

Заключение

React DevTools — это мощный инструмент для разработки и отладки React-приложений. Он позволяет исследовать иерархию компонентов, просматривать и изменять их props и state, а также анализировать производительность приложения. Регулярное использование React DevTools может значительно упростить разработку и помочь создавать более эффективные и надежные приложения.

Для получения более подробной информации о React DevTools, посетите [официальную документацию](#) или [репозиторий на GitHub](#).